

COMP1312 Programming I (UoSM) 2024/2025
Coursework I

Module Code:	COMP1312		
Module Title:	Programming I		
Module Leader:	Asst.Prof. Dr Fairuz Safwan Mahad		
Assessment Type:	Individual	Weightage:	20%
Submission Due Date:	22 nd November 2024, 16:00 (Malaysia Time)		
Method of Submission:	Blackboard		
GenAI Tier Specification	Tier 1: No GenAI use [applies All - Entire work]		
This assessment relates to the following Module Learning Outcomes:			
A. Knowledge and Understanding	A1. Basic programming constructs including sequence, selection and iteration, the use of identifiers, variables and expressions, and a range of data types A2. Good programming style in Python and C		
B. Subject Specific Intellectual and Research Skills	B1. Analyse a problem in a systematic manner B2. Design, implement, debug and test simple programs in Python and C		

COMP1312 Programming I (UoSM) 2024/2025 Coursework I

This Coursework is worth **20%** of the total marks for this module. The deadline is **22nd November 2024, 16:00 (Malaysia Time)**, handin via the blackboard site. The work is expected to take **8 to 12 hours**, can vary based on your familiarity with C and problem-solving skills.

Coursework Overview:

In this coursework, you must **demonstrate** a *Banking System Application* using *C programming*. You have the freedom to choose to work with any Integrated Development Environment (IDE). This exercise aims to emphasize fundamental programming concepts, such as how to make efficient use of the *various data types and variable types, arrays, loops, conditionals, functions, and user inputs*. You will write a program (as a deliverable of this coursework - I) that enables users to *create new bank account, perform deposit, withdrawals, remittance and delete bank account*.

Coursework Requirements

The essential requirement of this coursework is to demonstrate the *Banking System Application by using ONLY native C Programming features and methods, you are not allowed to use third party libraries*. The coursework requirements are as follows:

1. Basic Functionality

- Implement a text-based menu that repeatedly prompts the user to select from the available options.
- Allow users to perform **FIVE(5)** operations, **Create New Bank Account, Delete Bank Account, Deposit, Withdrawal and Remittance**.
- Ensure the program continues running until the user chooses to **exit**.

2. Create New Bank Account

- When creating a new bank account, prompt users to input details such as **Name, Identification Number, Type of Account (Savings or Current)**,
- Generate a **bank account number** by generating a random number between 10,000 and 1,000,000.
- Declare a variable for amount and initialize it with 0 by default.
- Store all the information above in a file (.txt format)
- One file represents one bank account (e.g. **TWO(2)** bank accounts will have **TWO(2)** files, 5471252.txt, 7904612.txt)
- All .txt files should be placed in a directory called **“database”**.

3. Delete Bank Account

- Retrieve all banking account numbers from the file and allow users to choose and delete from the current list of existing bank account number.
- After deleting a bank account, all information (current balance, personal information, banking number etc) should no longer be available in the file. The file should also be deleted from the **“database”** directory.

4. Deposit

- Allow users to deposit an amount into an existing bank account
- The current balance should be updated in the file after the deposit

5. Withdrawal

- Allow users to withdraw an amount from an existing bank account
- The current balance should be updated in the file after the withdrawal

6. Remittance

- Allow users to transfer an amount from an existing bank account to another existing bank account
- The current balance of both affected accounts should be updated after the remittance and the new current balance should be updated in the file respectively.
- Additionally, the following rules apply:
 - Savings to Current account will incur a 2% remittance fee (e.g. Transfer amount: RM100, remittance fee: RM2)
 - Current to Savings account will incur a 3% remittance fee (e.g. Transfer amount: RM100, remittance fee: RM3)
 - The remittance fee shall be deducted from the current balance of the sender

Important:

While no specific error handling/edge cases (examples are provided only for understanding) are mentioned in the coursework description, it is essential that ***your program is robust and handles unexpected or invalid inputs. You are responsible for considering and addressing potential edge cases to ensure the program does not crash or behave unpredictably with invalid inputs.*** Implementing proper input validation and error handling is crucial for a well-functioning program.

Submission Guidelines:

- You will put all of your source code and executable files in a folder and submit your work as a single ***.zip file ONLY via blackboard.***
- **Your source code should include:**
 - ***Clear and concise comments*** explaining the code's functionality and key points.
- Submit your **.zip** file in the following naming format - ***(banking_system_<student_id>.zip).***
- The ***code must run without errors*** and ***meet the functionality outlined in the coursework requirements.***

Academic Integrity

"In this entire coursework, students are **prohibited** from **using any GenAI tools** for their **assessed work**. This includes entering any part of the assignment or your assessed work to GenAI, whether by pasting/typing, uploading files, or describing content directly or through plugins. Basic tools that assist spelling and grammar, translation and calculation without generating new content or ideas, can be used unless specified otherwise by the assessment setter. GenAI may be used to explain lecture slides and notes to enhance understanding of a relevant topic areas. Students are not required to complete a GenAI Declaration Form."

For more information on GenAI refer [here](#)

The aim of this coursework is to promote learning. You are encouraged to discuss concepts with peers and refer to external resources. However, all source code must be your own original work. Use of AI-generated code (e.g., ChatGPT, Bard) will result in penalties according to university policies. Ensure your submissions reflect your own understanding and effort.

Help and Assistance

During lab hours, staff will be on hand to assist you. You will need to continue on your own schedule after the lab.

Marking Guidelines & Feedback

This coursework accounts for 20% of the total marks for this module and will be assessed according to the attached rubrics. Marks and written feedback will be provided within 3 weeks of the due date. Please note that the marks are provisional and subject to change after moderation.

Marking Guidelines

Grading Criteria	Wtg	Excellent 5	Above average 4	Average 3	Fair 2	Low 1	0	Max Mark (≈Wtg * Score)
Create New Bank Account	4	All required personal information are prompted and stored accurately in a file. Generated bank account number are all unique and not repetitive	All required personal information are prompted and stored accurately in a file with minor bugs. Generated bank account number are all unique and not repetitive	Some required personal information are prompted and partially stored in a file with minor bugs. Some bank account numbers are not unique	Some required personal information are prompted but not stored in a file. Some bank account numbers are not unique	Missing many required personal information and information partially stored in a file. No bank account number is generated	Feature not implemented.	20
Delete Bank Account	3	The bank account along with all the information are successfully and accurately deleted from the file without any trace.	The bank account along with all the information are successfully and accurately deleted from the file with minor bugs	The bank account along with some of the information are deleted from the file with minor bugs	Partial deletion of data with many bugs such as wrong account deleted	Deletion of banking account is performed but the file is not updated.	Feature not implemented.	15
Deposit	3	Deposit process is successfully, and accurately performed and current balance is updated in the file.	Deposit process is successfully, and accurately performed and current balance is updated in the file with minor bugs	Deposit process is successful, but current balance is not accurate. Current balance is updated in the file.	Deposit process is successful, but current balance is not accurate. Current balance is inaccurately updated in the file.	Deposit is performed but the file is not updated.	Feature not implemented.	15
Withdrawal	3	Withdrawal process is successfully, and accurately performed and current balance is updated in the file.	Withdrawal process is successfully, and accurately performed and current balance is updated in the file with minor bugs	Withdrawal process is successful, but current balance is not accurate. Current balance is updated in the file.	Withdrawal process is successful, but current balance is not accurate. Current balance is inaccurately updated in the file.	Withdrawal is performed but the file is not updated.	Feature not implemented.	15

Remittance	4	Remittance process is successfully, and accurately performed and current balance is updated in the file.	Remittance process is successfully, and accurately performed and current balance is updated in the file with minor bugs	Remittance process is successful, but current balance is not accurate. Current balance is updated in the file.	Remittance process is successful, but current balance is not accurate. Current balance is inaccurately updated in the file.	Remittance is performed but the file is not updated.	Feature not implemented.	20
Error Handling and Validation	3	Handles all invalid inputs and edge cases flawlessly. Program never gets stuck or produces unexpected behaviour due to bad input.	Most input errors and edge cases are handled, but the program may have minor issues with unusual inputs.	Basic validation present, but program behave unexpectedly due to bad input.	Little error handling or validation, frequent issues with invalid input, causing the program to behave incorrectly.	No input validation or error handling, program fails to handle invalid input effectively.	No error handling implemented.	15
Total	20							100

Any work submitted after the deadline's time will be subject to the standard University late penalties unless an extension has been granted, in writing by the Senior Tutor, in advance of the deadline. Details on the University's late penalties can be found here:

- <https://www.southampton.ac.uk/~assets/doc/quality-handbook/Late%20Submission.pdf>

Edge Cases (This will be hidden to students)

In the context of a Banking System program, here are a few examples of "bad inputs" or edge cases that the students' code should be able to handle:

1. Non-numeric Input

When users are expected to input number (e.g., deposit money), they might enter something that isn't a number:

- **Edge Case:** The user enters a string instead of a number.
 - **Bad Input:** "four hundred"
 - **Expected Handling:** The program should catch this and ask for valid numeric input instead of crashing.
 - **Example Error Message:** *"Invalid input. Please enter a number."*

2. Insufficient Funds

When users withdraws/remit money, there could be an insufficient amount of money in the current balance:

- **Edge Case:** The user enters an amount to withdraws/remit
 - **Bad Input:** entering 400 but the current balance is showing 100.
 - **Expected Handling:** The program should notify the user that there is an insufficient amount.
 - **Example Error Message:** *"Insufficient funds. Please try again."*

3. Invalid Option in Menu

When selecting from the menu, the user might input an invalid option.

- **Edge Case:** The user enters a number or character that is not part of the available menu options.
 - **Bad Input:** Typing 7 when the menu only has options 1–5.
 - **Expected Handling:** The program should display an error message and re-prompt the user.
 - **Example Error Message:** *"Invalid option. Please select a valid menu option."*

Handling these bad inputs gracefully ensures that the program doesn't crash or behave unexpectedly, and instead informs the user of the mistake and prompts them for correct input.